EXTENDED ESSAY

Topic: Effect of Loss Function Choice on CNN Age Estimation from Retinal Fundus Images

Research Question:

To what extent does the choice of loss function affect CNN performance in age prediction from retinal fundus images?

Subject Area: Computer Science
Report word count: 3958
Session: May 2027

Student Code: TBD-PLACE-HOLDER

Introduction....3
Background....4
AI, ML and DL....4
Neural Networks (NN)....4
CNN....5
Learning & Loss (Cost) functions: MAE, MSE and Cross-Entropy....6
Previous Investigations....8
Hypothesis....9
Methodology....9
Patient Data for Training, Validation and Test....9
Model....9
Experiment....10
GPU Server & Config....12
Client Dev Machine Config & Source Code....12
Data Collection....12
Quantitative....12
Qualitative Observation....13
Data Processing and Analysis....13
Conclusion....22
Evaluation....23

Introduction

In the field of retinal imaging, the images used are retinal fundus photographs, the non-invasive pictures of the back of the eye. They show the retina, the optic disc, the macula, and the retinal blood vessels, and often reflect changes in a person's health. Figure 1 shows the retinal fundus photographs taken by Airdoc-AIFUNDUS equipment from Airdoc Corp.

Figure 1: Retinal Fundus Photographs

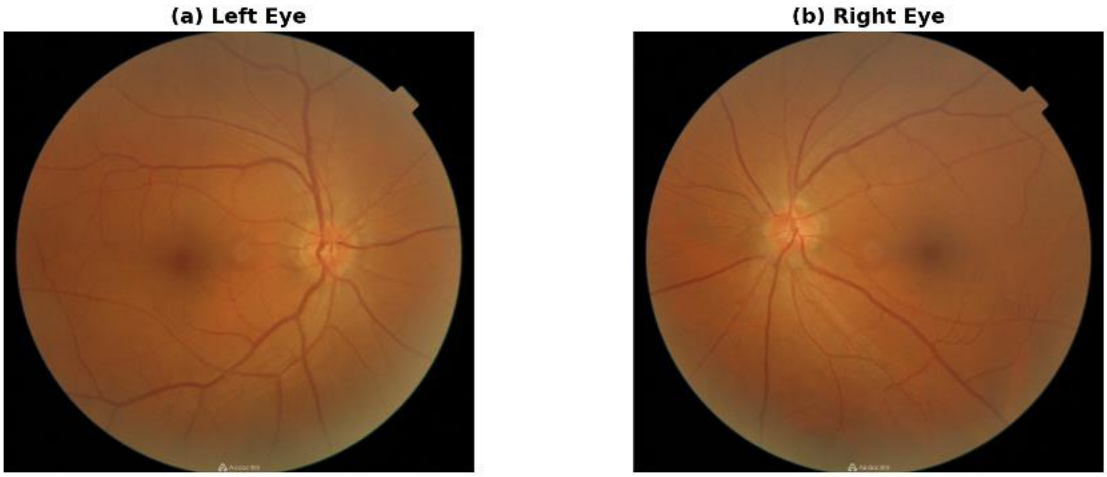


Figure 1. Example retinal fundus photographs from the left and right eye: the overall red orange circular area is the retina; the bright circular region is the optic disc; the darker central area is the macula, and the branching lines are retinal blood vessels.

Aging affects the body in visible ways: blood vessels may become narrower, tissue may change in appearance, and certain disease risks increase with age. Previous studies suggest that deep learning neural networks (Goodfellow) can estimate age and cardiovascular risk factors from retinal images, as the eye may contain patterns linked to biological and chronological aging (Poplin et al., 2018; Zhu et al., 2023). This is important because age prediction is not only about computing a number – if a model predicts an age that is consistently older or younger than the patient's real age, that gap is worth studying as a biomarker of health or disease risk.

In this experiment, I'll explore the effects of using different loss formulations (Mean Absolute Error, Mean Squared Error and Cross Entropy) on deep learning model's age prediction in the field on retinal imaging.

Background

AI, ML and DL

Artificial Intelligence (AI) refers to the broad field of computer science to train computers to imitate human intelligence to solve problems such as reasoning, problem-solving, perception, language understanding, and decision-making. Machine Learning (ML) is a major subfield of AI that focuses on enabling computers to learn data and perform tasks using algorithms and models instead of using hard-coded rules (IBM Data and AI Team). Deep Learning (DL) is a subset of Machine Learning that utilizes multi-layered artificial neural networks (ANN).

Neural Networks (NN)

Artificial Neural Networks (ANNs), also called neural networks (NN), are at the heart of deep learning. Inspired by human neural networks, a typical neural network consists of interconnected nodes in layers: an input layer, one or more hidden layers, and an output layer.

A neural node is illustrated in Figure 2a, it takes an input vector X ($x_1, x_2, x_3, \ldots x_n$), has a node specific bias value (b) and maintains a weight vector $\vec{w}$ storing w_i for each x_i. The bias (b) decides the activation threshold value of the node, while w_i decides the level of importance of input x_i. A neural node does two functions (Google for Developers):

1. Summation (z) of all the $x_i * w_i$ and the bias (b),

$$z = x_1 * w_1 + x_2 * w_2 + \ldots + x_n * w_n + b$$

2. Activation with the summation value z as input, going through an activation function $f(z)$, and a as the output

$$a = f(z)$$

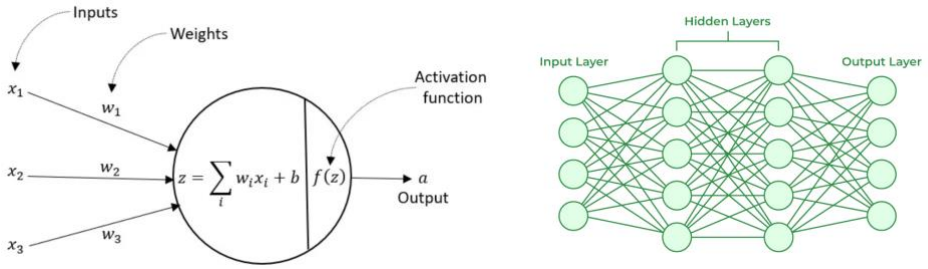


Figure 2a. A neural node in a NN (Agarwal) *Figure 2b. An example NN ("Artificial")*

The activation function $f(z)$ adds non-linearity to the signal, just like the selective activation mechanism of a human neuron. There are a variety of such functions, some of the common ones include (Google for Developers, "Neural Networks"; Google):

- Sigmoid, $f(z) = \frac{1}{1+e^{-x}}$, with a range of (0,1);
- ReLU: $f(z) = 0\ when\ z < 0;\ f(z) = z\ when\ z \geq 0$,with a range of [0, ∞) ;
- Softmax, $f(z_i) = \frac{e^{z_i}}{\sum_{k=1}^{N} e^{z_k}}$, where *k*=1, 2, ... N, N is the number of output classifications, Softmax has a range of (0,1).

Figure 2b illustrates a NN with 1 input layer, 2 hidden layers and 1 output layer. The input layer takes raw input and passes the information (features) to the first hidden layer. Data flows through each layer, through each node's summation and activation, layer after layer, until the final output. A hidden layer computes on the features passed from the previous layer (input layer or hidden layer) and transfers the result to the next layer (a hidden layer or output layer). The output layer processes the information learned through the previous hidden layer and delivers the result. This process of passing data from one layer to the next layer defines this NN as a feedforward network.

CNN

A CNN is a class of neural network widely used in imaging recognition (LeCun et al., 1998). As illustrated in Figure 2 blow, it consists of a feature extraction stage, in which convolutional and pooling layers detect progressively more abstract patterns, and a classification stage, in which fully connected layers combine those features to produce a prediction.

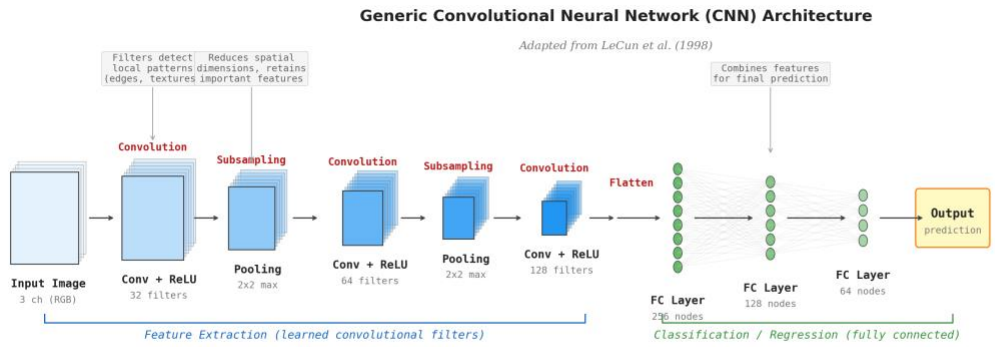


Figure 3: how a CNN processes an image before producing a prediction, adapted illustration (LeCun et al., 1998)

Learning & Loss (Cost) functions: MAE, MSE and Cross-Entropy

A NN Loss function calculates the Loss (also called Error), or Cost $\boldsymbol{C}$, using the prediction outputs against the training example's labelled results (target vector y). It is through iterations of minimizing the Loss that a NN learns through training its weights and biases (Google for Developers, "Training using Backpropagation") against a labelled dataset:

a. Initialize all nodes' weights with random values and biases with 0s.

b. Feed forward the training data through the NN to compute outputs.

c. Calculate the Loss C, which is a function of the weights and biases.

d. Use mechanisms called gradient descent ("Gradient descent") and back propagation ("Backpropagation") to tune the NN's weights and biases to minimize the loss and improve precision of predictions.

e. Repeat steps a to d until achieving maximum precision and minimal loss, which usually takes many epochs. An epoch is training the NN with all the training data for one cycle.

However, to prevent the model from simply memorizing the training data (overfitting), the dataset is split into three parts. After each epoch, the model runs step (b) on a separate Validation Set —but crucially, it does NOT perform step (d) on this data. Instead, it calculates the Validation Loss/Error just to monitor performance. If the Training Loss keeps decreasing but the Validation Loss starts increasing, this signals overfitting, and training should be stopped

immediately (a technique called "early stopping"). The model's weights are only ever updated using the Training Set.

Finally, after all epochs are complete and the best model is selected based on the Validation Loss, the model is evaluated one last time on a completely unseen Test Set. This produces the final Test Error, also called Prediction Error, the single most important number, as it represents the model's true real-world precision. The Test Set is used exactly once, at the very end, and never during the training or validation process.

Loss function is vital for the learning of Neural Networks; therefore, it makes sense to study what effects the choice of the loss function might have on the performance of the NN. In the field of AI Medicine, there are some popular loss functions, such as Mean Absolute Error (MAE), Mean Squared Error (MSE), and Cross-Entropy Classification (CE).

Suppose a NN is used to train and predict people's ages using their retinal images.
If the real age is y_i, and the predicted age is $\hat{y}_i$, for sample size n, then the Mean Absolute Error loss (Goolge for Developers, "Linear Regression") is:

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

In MAE, each prediction error is penalized linearly. A 10-year error is treated twice as bad as a 5-year error.

The Mean Squared Error is (Goolge for Developers, "Linear Regression"):

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

in MSE, larger errors are penalized much more. For example, a 10-year error contributes four times as much loss as a 5-year error. Because of this, MSE may be better at reducing outliers, but more sensitive to noisy labels.

For cross-entropy classification (CE), the continuous age variable is first discretized into bins, each representing a distinct class. The neural network outputs a vector of raw scores, which is then passed through a softmax function to produce a probability distribution over all age bins. If the age class is k, then the CE loss for one single sample is defined as (TensorFlow):

$$CE = -\log(p_k)$$

Where p_k is the predicted probability of the correct age bin after softmax normalization. The softmax function converts these logits into a probability distribution as follows:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{C} e^{z_j}}$$

Where z_k is the raw scalar output score by the neural net for class k (total C class bins). The exponential function ensures all $p_k>0$, and the denominator normalizes the values so that:

$$\sum_{k=1}^{C} p_k = 1$$

In vectorized form, *CE* can also be written as:

$$CE = -\sum_{c=1}^{C} y_c \log(p_c)$$

Where C is the total number of age bins, y_c is a one-hot encoded vector (with a single 1 at the true class index c and 0 elsewhere, corresponding to the correct age bin), and p_c is the softmax probability for class c (Google for Developers, "Deep Neural Network Models"). This method is interesting because it changes the problem from predicting one number to choosing among age categories.

Previous Investigations

While many high-performing models exist (Poplin et al, Ahadi et al, Ninomiya et al), few studies systematically compare different loss functions (e.g., MAE vs. MSE vs. CE) on the same dataset and architecture for retinal fundus age estimation. Most papers select one loss function based on convention rather than empirical comparison.

Research Question: To what extent does the choice of loss function affect CNN performance in age prediction from retinal fundus images?

Hypothesis

MSE will produce the best overall predictive performance with the fastest convergence, owing to its quadratic penalty for the large errors. Mean Absolute Error (MAE) is expected to perform comparably on average but exhibit greater robustness to outliers and seed variability due to its linear nature. Cross-entropy (CE) classification is likely to underperform because it needs to round the predicted age number to a bin (e.g., exactly 42.0 years) -- Even a perfect classification (e.g., the predicted value is 42.5 years) will be wrong by up to ±0.5 years purely because of this rounding.

Methodology

This investigation compares three loss functions' effects on the same neural network. The model takes a retinal image as input and outputs either a continuous predicted age or a classified age bin, depending on the loss function. The independent variable is therefore the loss function: Mean Absolute Error (MAE), Mean Squared Error (MSE), or Cross-Entropy Classification (CE). The dependent variables are performance metrics on unseen data, especially mean absolute error values for train, validation and test/prediction runs.

Patient Data for Training, Validation and Test

The proprietary dataset consists of 28,030 masked retinal fundus photographs with patient age labelled from 19 to 87, provided by a clinical partner through AirDoc Corp. Images were split per round into 70:15:15 train:validation:test ratio, or images 19,622 | 4,204 | 4,204.

Model

The neural network chosen for this study is EfficientNet-V2-M (Tan and Le, 2021), a derivative of the convolutional neural network (CNN). It balances accuracy and efficiency well (Tan and Le, 2021). Indeed, AirDocAI already uses EfficientNet-V2-M as its main backbone for retinal-image prediction tasks.

Experiment

First, as the retinal images come in huge and slightly different and the network only takes in one size: 299×299, so a retinal fundus image is first preprocessed through a resize-blur-resize pipeline with min-max normalization, which does three jobs: 1) shrink them in two steps (897 → 598 → 299); 2) Blur a little before the last shrink, knocking off tiny camera grain and JPEG speckles; 3) stretch brightness so a dark photo and a bright photo both sit on the same 0–1 scale.

Next, the 299x299 image is passed through an EfficientNet-V2-M backbone (with its usual 1000 classes' classifier removed) to extract a 1,280-dimension feature vector. A customized regression (with MAE or MSE loss) or classification (with CE loss) head produces the final age prediction. APPENDIX-1 has the implementation details on the switching logic. Figure 4 illustrates the architecture of the neural network used.

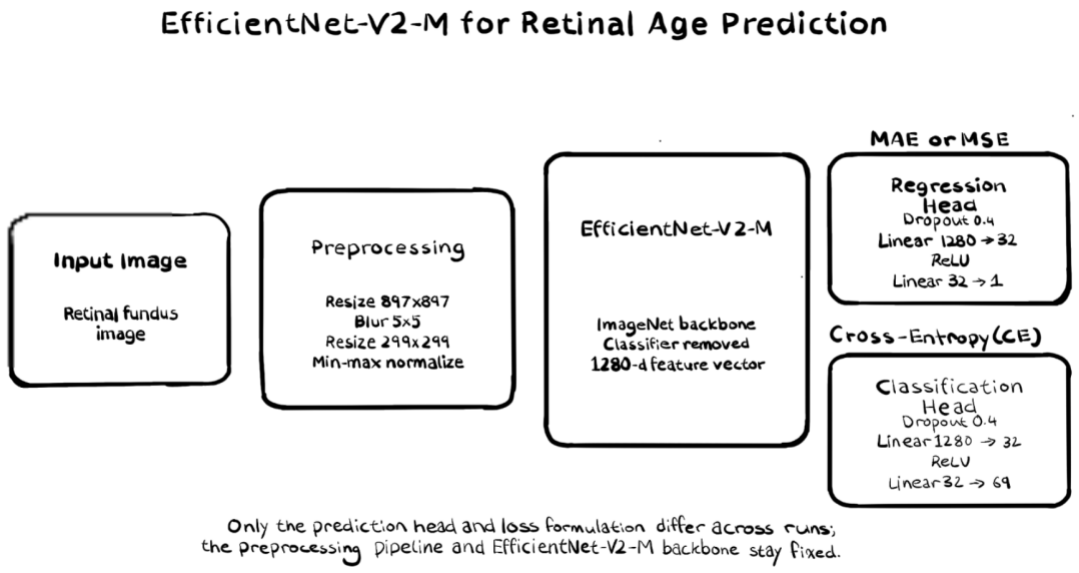


Figure 4: Architecture of the neural network used in all experiments. (Hand-Drawn)

With the same image inputs and training/validation/test parameters, the only variables between experiments are the loss formulation (MAE, MSE, or CE).

Next, three rounds of experiments were performed. In each round, the same neural net was trained once with MAE, once with MSE, and once with CE. This produced 3x3=9 main runs in total. This repeated-run design matters because deep learning results can vary due to random initialization (Brownlee). An earlier single-run comparison raised the possibility that an observed

difference between losses might come from randomness rather than from the loss function itself. All other training hyperparameters are held constant across the nine runs:

Optimizer: Adam

Initial learning rate: 0.006

Batch size: 20

Max epochs: 120

Early-stopping patience: 13 epochs

Weight initialization: Kaiming normal (fan_out, ReLU)

Random seeds: three, distinct across rounds, kept the same within a round

CE age binning (CE runs only): 69 bins, width 1 year, range [19, 87]

The entire three-loss comparison was repeated in three independent rounds:

- Round 1: seed = 42 → outputs001/{mae,mse,ce}-runs
- Round 2: seed = 43 → outputs002/{mae,mse,ce}-runs
- Round 3: seed = 44 → outputs003/{mae,mse,ce}-runs

Within each round, the three loss conditions shared the identical train/val/test partition and the identical random seed (an initial value used to control the "random" operations) for weight initialization. Consequently, any performance difference observed within a round can be attributed solely to the loss formulation, while differences across rounds capture the combined effect of initialization due to different seed numbers. Each round outputs a list of files that contain epoch, prediction error/loss, training metrics, etc, which we will collect for analysis.

A run script was used to launch the 3 rounds of tasks in turn; the essence of the script content and the training parameters are in APPENDIX-I.

GPU Server & Config

This project required GPU because training the NN (EfficientNet-V2-M) on thousands of retinal images would be too slow on a laptop. Experiments were run on a AirDoc's GPU cluster server with:

- 8 × NVIDIA GeForce RTX 3090 (24 GB VRAM each, 192 GB total), CUDA 11.4, driver 470.63.01,
- Ubuntu (Linux) 18.04 kernel 4.15.
- The CPU was a dual-socket AMD EPYC 7542 (128 logical cores) with 503 GB RAM.
- Storage was on a high-bandwidth network filesystem mounted at /mnt/workdir.

The cluster server is in a DMZ, accessible through a jump Windows Server containing two network cards, once facing the company LAN, the other facing the DMZ network.

Client Dev Machine Config & Source Code

A Mac M4 with 24GB RAM and 1TB disk. VS Code, python3, MS Remote Desktop, MS Office for Mac. Appendix-I has information for the source code repository.

Data Collection

The whole set of experiments was run, and the main experimental data from three rounds of training runs was collected. In each round, the same architecture was trained three times, with MAE, MSE, and CE. Therefore, a total of 9 runs' data has been collected.

Quantitative

For each run, the following results (9x) have been collected, after the security approval by Airdoc: (see APPENDIX-II to V for sample of the below files)

- training_log.txt
- epoch_metrics.csv
- val_predictions.csv
- test_predictions.csv

Qualitative Observation

- Peak per-GPU memory during training was 9.46 GB;
- training throughput was approximately 30 samples / second (≈ 1.5 batches / second at batch size 20).
- Average epoch duration (training plus validation) was approximately 13 minutes; total wall-clock time across the nine runs was approximately 132 hours.
- The total logged training time across the three main rounds is about **121.09 GPU hours**.
- The training script allowed up to 120 epochs, but the runs did not usually reach 120 because early stopping (13 consecutive non improvement loss) was enabled. The logs show that the runs stopped at different points depending on the loss and seed. This is useful because it gives information about convergence and stability.
- Table 1 shows the run times for the 3x3 combinations as well as the totals. Note that For round 1's CE run, the training collapsed early after just 3.5 hours, which will be discussed in the analysis section.

Round	MAE (h)	MSE (h)	CE (h)	Round Total (h)
1	18.0	12.4	3.5*	33.9
2	11.7	10.8	14.6	37.1
3	17.7	15.2	17.3	50.2
Total	47.4	38.4	35.4	121.2

Table 1. Training time (hours) per experimental round and loss formulation. (-collapsed run)*

Data Processing and Analysis

The quantitative result files were programmatically parsed and interested fields such as epoch, prediction error values per epoch for the train and validation per loss function were saved into data files round_{1,2,3}_raw.csv, and the prediction results summarized in runs_data.csv (APPENDIX-VI). Table 2 lists partial content of round_1_raw.csv for the train and validation phases of round 1:

Epoch	MAE train	MSE train	CE train	MAE val	MSE val	CE val
1	12.00	11.88	12.70	10.80	18.97	13.56
2	11.15	11.34	12.72	10.60	155.36	13.15
3	10.09	11.39	12.80	9.39	22.48	12.08
4	9.08	11.17	12.84	8.51	457.19	12.08

5	8.73	11.24	12.69	8.47	11.26	13.00
6	8.45	10.20	12.76	7.74	8.22	13.00
7	8.25	8.41	12.83	8.28	9.05	12.50
8	8.12	7.76	12.81	7.56	8.94	12.08
9	7.60	7.28	12.90	6.34	9.60	12.08
10	6.16	6.89	12.82	5.56	6.21	12.08
11	5.99	6.73	12.39	5.26	5.98	12.08
12	5.82	6.44	12.25	5.31	5.54	13.00
13	5.69	6.22	12.62	5.53	5.93	13.00
14	5.64	6.08	13.07	5.05	8.03	13.00
15	5.56	5.93	13.07	5.30	5.70	13.00
16	5.51	5.80	13.07	4.88	8.54	13.00
17	5.52	5.73		5.48	7.75	
18	5.43	5.61		5.08	5.91	
19	5.33	5.57		4.95	5.02	
20	5.23	5.42		5.35	5.63	
...	...	...				

Table 2. Processed Train and Validation Prediction Error data for round 1 with MAE, MSE, CE loss modes.

MSE exhibits highest variations in early epochs before stabilizing, CE shows lowest variations.

The training and validation prediction errors, mean |true age – predicted age|, were plotted against epochs for each loss mode per round.

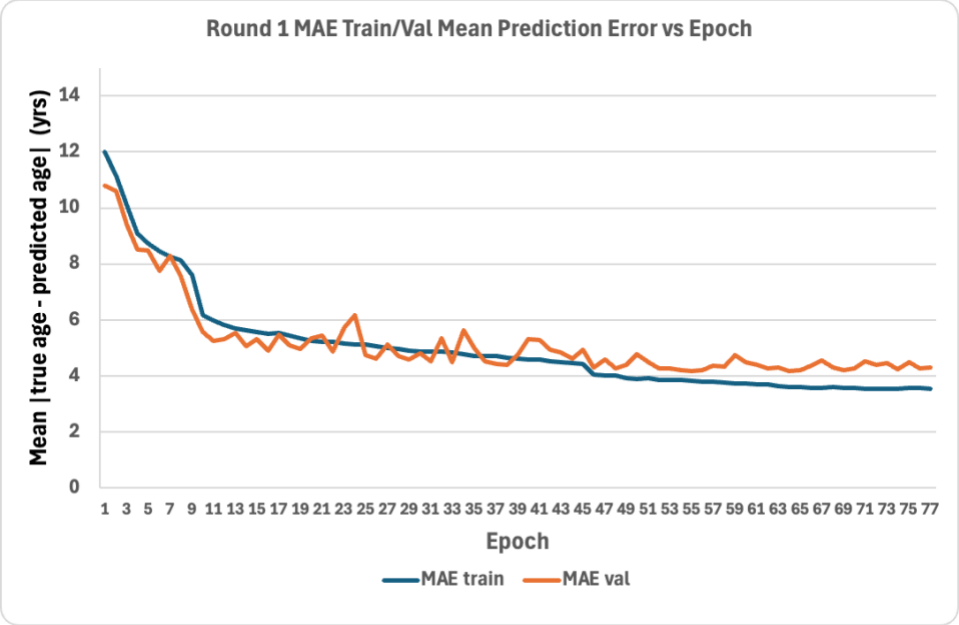


Figure 5A. MAE Learning Prediction Error vs Epoch, round 1. Round 2 and 3 show similar patterns.

Figure 5A shows MAE round 1. The training and validations prediciton errors start with 12.00 and 10.80 at epoch 1, then sharply reduce and stabilize at around epoch 65. Post epoch 65 the training enters the phase of "overfitting", since there are 13 consecutive validation prediction errors that are worse than that at epoch 64, triggering early stop.

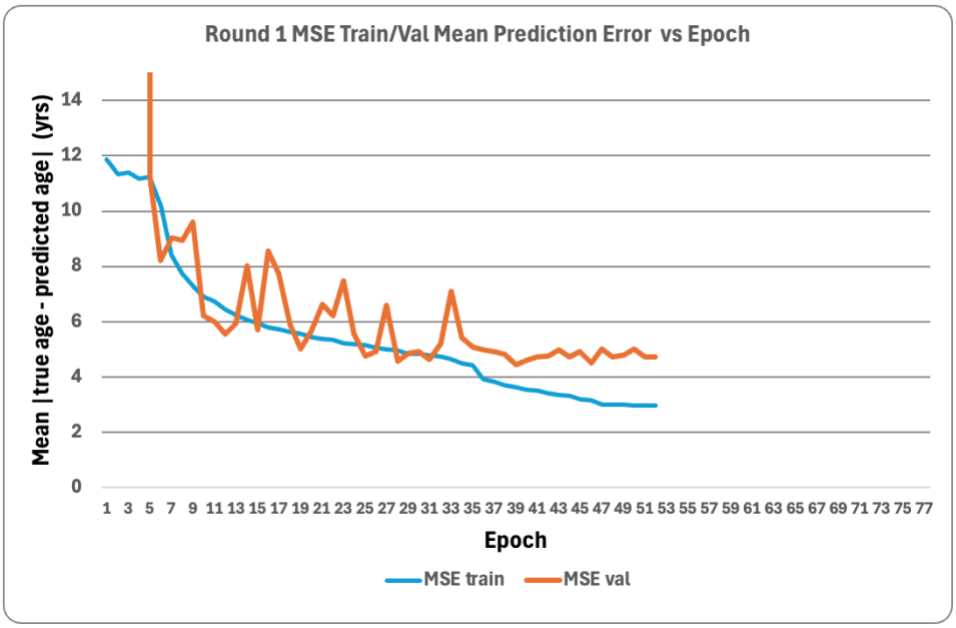


Figure 5B. MSE Learning Prediction Error vs Epoch, round 1. Round 2 and 3 show similar patterns. For epoch 1-4, its validation prediction errors (18.97, 155.36, 22.48, 457.19) were too high to be within the plotted area.

In Figure 5B, MSE round 2 has high validation prediction errors at epoch 1-4, and then roughly stabilizes at around epoch 39, with relatively large variations (difference between orange and blue) in between. The overfitting phase is from epoch 40-52.

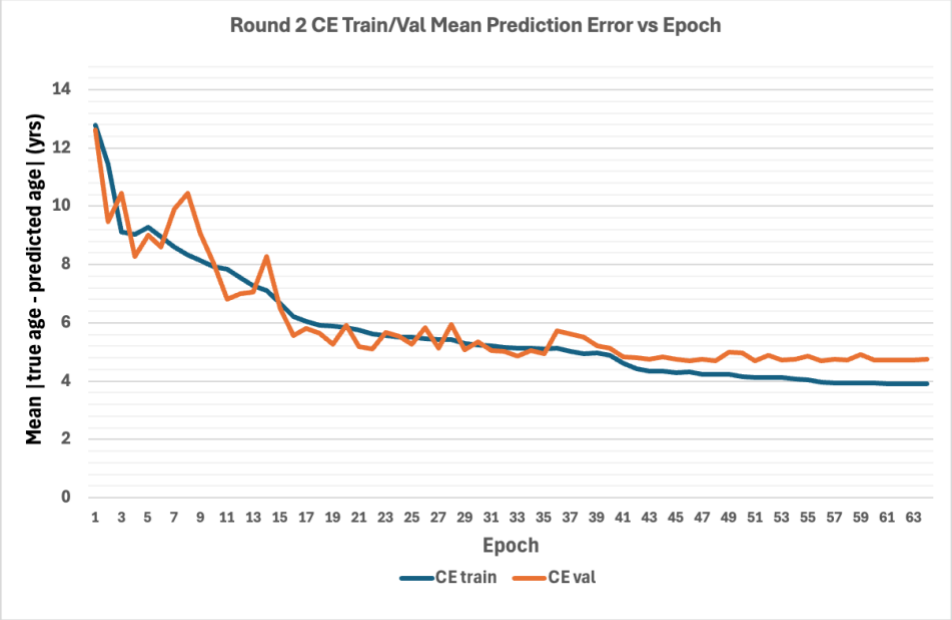


Figure 5C. CE Learning Prediction Error vs Epoch, round 2. Round 3 is similar. (Round 1 is early collapse.)

Figure 5C shows that CE round 2 exhibits relatively low validation prediction errors throughout, with post-epoch 50 as the overfitting phase.

MSE training converges faster than MAE training, while CE mode shows sensitivity to initialization

A summary of the 3x3 runs based on the data files above is in Table 3. All runs finished training on early stops. MSE runs finished training at epochs 52, 46, 62, compared to MAE's finish at epochs 77, 52, 76. Timewise, Table 1 shows that MSE takes less time than MAE at each round, and collectively takes 38.4 hours for all three runs, less than MAE's 47.4 hours. Therefore, MSE converges faster than MAE, suggesting that MSE's quadratic approaches accelerate the converging faster by penalizing more on the errors.

Round	Loss mode	Training collapsed?	Training epochs
1	CE	**TRUE**	16
2	CE	FALSE	64
3	CE	FALSE	77
1	MAE	FALSE	77
2	MAE	FALSE	52
3	MAE	FALSE	76
1	MSE	FALSE	52
2	MSE	FALSE	46
3	MSE	FALSE	62

Table 3. Summary of the three rounds of experiments.

8 out of the 9 runs were normal, except that Round 1 CE was classified as collapsed because its validation error remained between 12.08 and 13.00 years and did not improve from epoch 4 to epoch 16, resulting in early stop of training after epoch 16 (see APPENDIX_II). It is not surprising that this resulting model produced the identical prediction value of 32.00 (see APPENDIX_IV) for all test images, with a 12.04 Prediction error. The collapsed round-1 CE run suggests that the CE mode was sensitive to initialization. This is consistent with the broader reproducibility concerns raised by Henderson (Henderson et al., 2018) that deep-learning results can change because of random weight initialization, data order and sampling, hyperparameter choices, etc.

MAE and MSE show comparable Prediction Errors, while CE shows higher Prediction Errors.

Plotted in Figure 6 is the prediction error/loss data, also listed in Table 4. For round 1 and round 3, MAE has lower prediction errors of 4.05 and 4.28 (years) than MSE's 4.39 and 4.55 years; in round 2, MAE's 4.61 years is higher than MSE's 4.41 years. In other words, MAE and MSE have shown comparable prediction errors across the three rounds. CE's prediction errors in round 2 and 3 (round 1 excluded due to collapse) are 4.79 and 4.87, quite higher than all three rounds' MAE and MSE prediction errors.

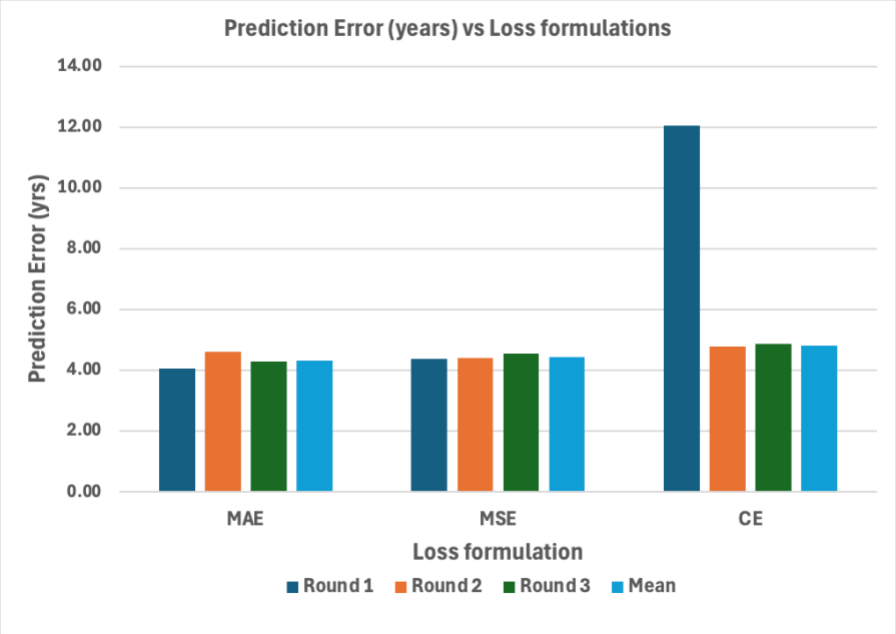


Figure 6. Prediction Error vs Loss modes.

Collectively, MAE has the lowest mean prediction error of 4.31±0.28, MSE with a comparable 4.45±0.09, and CE much higher with 4.83±0.06. MAE has the highest standard deviation(SD) of 0.28, while MSE and CE have relatively low SDs of 0.09 and 0.06. Such variations can be traced back to their definitions. For MAE, any shift in age distribution directly shifts the numeric error linearly, resulting in higher SD. For MSE, any shift in elderly patients causes massive, squared error which overreacts and biases the model to the mean, making it artificially consistent, resulting in low SD. For CE, the loss is capped and compressed by its log function, $CE = -\log(p_k)$. So, even if the validation set shifts, it stays in a narrow band of lower SD.

Round	MAE (yrs)	MSE (yrs)	CE (yrs)
1	4.05	4.39	12.04*[1]
2	4.61	4.41	4.79
3	4.28	4.55	4.87
Mean	4.31	4.45	4.83*[2]
SD	0.28	0.09	0.06

Table 4. Test Set Prediction Error data for the three Loss formulations.

*[1] *The run collapsed during the training/validation phase of round 1 experiment.*

*[2] *The mean calculation used round 2 and 3's values, excluding round 1 CE's value since it collapsed.*

All three loss modes struggle in prediction of >=60 age group, with CE being the worst.

To further find out how the three loss modes affect the prediction errors of different age groups, the experimental results were regrouped with the age groups in Table 5. Round 1 had higher errors for CE than MAE and MSE for all age groups, with unusually high errors for age group ">=45 and <60" and >=60, due to the early collapse.

Round	Age group	MAE	MSE	CE	n
1	<30	3.11	3.74	5.34	890
1	>=30 and <45	3.59	3.94	4.69	1783
1	>=45 and <60	4.33	4.70	19.86	1079
1	>=60	7.08	6.73	35.55	452
2	<30	4.04	3.57	3.41	914
2	>=30 and <45	3.87	4.05	3.80	1710
2	>=45 and <60	4.93	4.70	4.11	1118
2	>=60	7.69	6.72	12.83	462
3	<30	3.01	3.86	3.53	953
3	>=30 and <45	3.62	4.13	3.99	1703
3	>=45 and <60	4.94	4.89	4.18	1081
3	>=60	7.77	6.75	12.41	468

Table 5. Test Data (pooled round 1, 2, and 3 test splits) Age Distribution.

All rounds show higher prediction errors for age group >=60 for all loss modes, with CE particularly the worst (12.83). Round 2 and 3 show similar patterns for all age groups. Figure 7 plots round 2 of three loss modes' Prediction Errors.

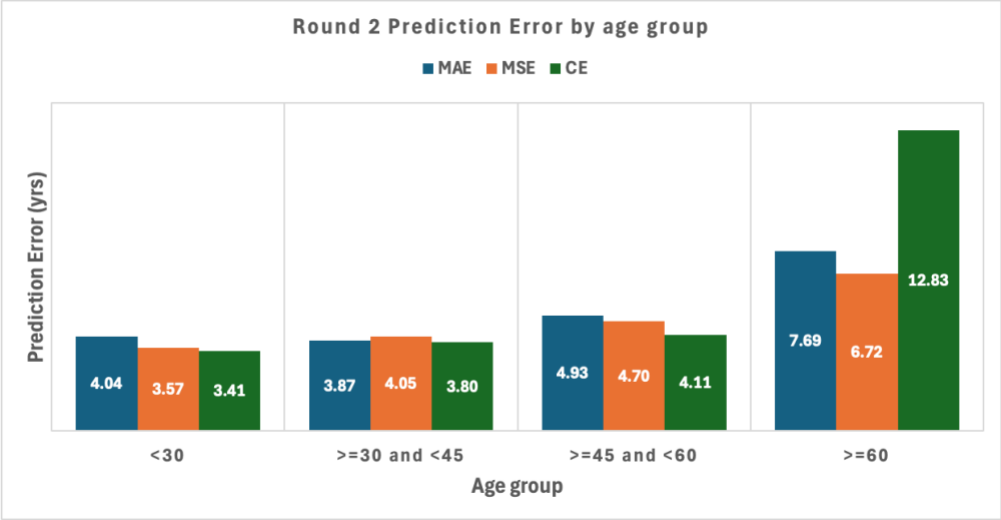


Figure 7. Prediction Error by age groups, round 2. Round 3 is similar. Round 1 is excluded for its collapse.

To further study why all the loss modes' performance across age groups, we grouped the Test Data's true age by groups in Table 6. The train and validation sets' exact ages are not available due to Airdoc's data security policy, but we can safely assume the overall dataset follows a similar distribution.

Age Group	Number	%	Mean	Median
ALL	12613	100.00	41.65	38.39
<30	2757	21.86	26.62	26.92
>=30 and <45	5196	41.20	36.31	35.80
>=45 and <60	3278	25.99	51.83	51.59
>=60	1382	10.96	67.57	65.80

Table 6. Test Data (pooled round 1, 2, and 3 test splits) Age Distribution.

Figure 8 is a pie chart from Table 6 data. All three loss modes made larger errors on patients aged 60 and above. The test set is young majority. Ages run from about 19 to 87, but the mean is 41.65 years, and the median is 38.39. Only about 11% of test images are from patients aged 60 or older, and that group is also the widest (60–87 years). An imperfect predictor then shrinks toward the majority age: young patients are over-predicted and older patients are under-predicted

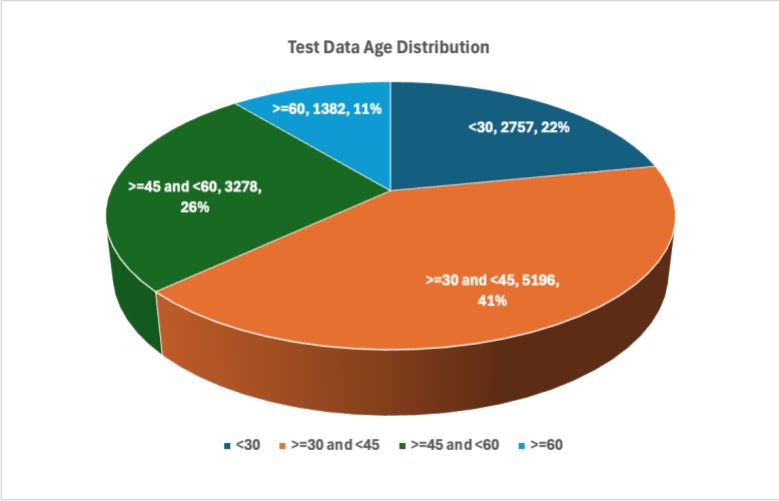
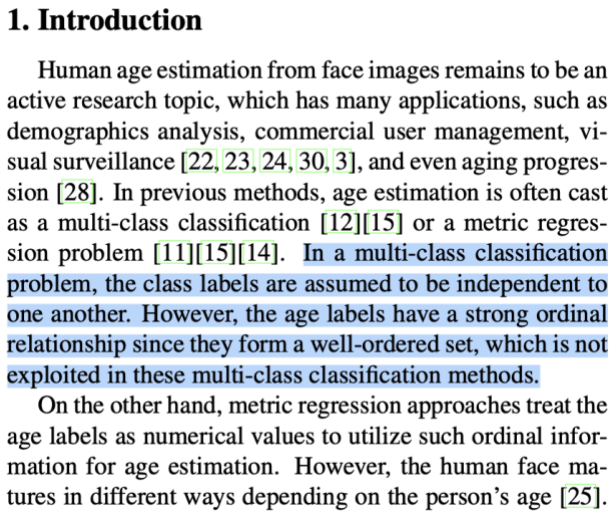


Figure 8. Test Data Age Distribution.

CE is worst on age group >=60. A likely reason is that CE is classification, not regression. The samples' ages range from 19 to 87, split into 69 one-year bins. Only ~11% of images are 60+, spread across 28 bins, so those classes barely appear in training. MAE and MSE still know that 64 is close to 65, but CE does not explicitly encode the ordinal distance between different classes so predicting 65 instead of 64 is not treated as a smaller error than predicting 65 instead of 30.

CE is competitive under 60, where bins are common, and fails on 60+ because those years are rare, unordered classes. For CE, categories are unrelated (e.g., cat, dog with on, car), but ages have an ordering and that CE treats ages such as 20 and 21 as just different classes like 30 and 67, even though 20 and 21 are very similar. Our findings corroborate previous findings that cross-entropy classification ignores the ordinal structure of age labels, often resulting in inferior optimization, prediction accuracy, and stability (Niu et al.; Rothe et al.). Niu states that treating age bins as unrelated classes is a bad ordinal model, explains why plain CE age classification is a bad fit for age, and that age should be treated as ordered, not as unrelated labels.



1. Introduction

Human age estimation from face images remains to be an active research topic, which has many applications, such as demographics analysis, commercial user management, visual surveillance [22, 23, 24, 30, 3], and even aging progression [28]. In previous methods, age estimation is often cast as a multi-class classification [12][15] or a metric regression problem [11][15][14]. In a multi-class classification problem, the class labels are assumed to be independent to one another. However, the age labels have a strong ordinal relationship since they form a well-ordered set, which is not exploited in these multi-class classification methods.

On the other hand, metric regression approaches treat the age labels as numerical values to utilize such ordinal information for age estimation. However, the human face matures in different ways depending on the person's age [25].

Figure 9. Niu et al states that age should be treated as an ordered-set instead of equal categories, consistent with our findings.

Conclusion

The experiments demonstrate that the choice of loss formulation affects the predictive performance of an EfficientNet-V2-M convolutional neural network for chronological age estimation from retinal fundus images in a differentiated way.

Across three independent rounds that controlled for random seed, Mean Absolute Error (MAE) and Mean Squared Error (MSE) produced closely comparable test-set Prediction Errors (overall means 4.31 ± 0.28 years and 4.45 ± 0.09 years, respectively). My hypothesis that MSE will have the best predicative performance is proven to be incorrect, since MAE has the lowest error. Therefore, converging fast does not necessarily lead to better prediction.

In contrast, the cross-entropy (CE) classification formulation produced a clear increase in error (4.83 ± 0.06 years when the collapsed run is excluded) together with markedly greater training instability, including one collapsed run.

MSE consistently reached early-stopping criteria in fewer epochs and required less total training time than MAE, supporting the hypothesis that its quadratic penalty accelerates convergence.

CE underperformed especially on the sparsely populated ≥60 age group, likely because the conversion of continuous ages into unordered one-year bins discards ordinal structure and leaves rare classes under-represented.

All three formulations showed elevated errors on patients aged 60 and older, reflecting the young-skewed age distribution of the 28,030-image dataset (mean 41.65 years, only ≈11 % ≥60).

Combined, the results indicate that switching between MAE and MSE has only a modest effect on age-prediction accuracy, while the adoption of plain cross-entropy classification incurs a clearer penalty in both accuracy and training reliability. The findings underscore the importance of multi-seed experimental design when comparing loss functions in deep learning and suggest that ordinal-aware alternatives to standard CE may be preferable when age is treated as an ordered variable.

Evaluation

The experimental design supports the internal validity of the comparison. The independent variable was isolated by controlling the backbone, image preprocessing pipeline, optimizer, learning-rate schedule, batch size, early stopping patience, and train/validation/test partitions constant within each round; only the loss function and the associated output differed.

Doing the three-loss comparison under three distinct seeds allowed systematic effects to be detected from initialization noise (Henderson et al., 2018). Per-epoch train and validation metrics, together with final test predictions, provided transparent, reproducible evidence for the reported means, standard deviations, and convergence behavior.

However, there are limitations that future improvements can be made upon:

- Data reliability: Bound by the fact that the patient dataset which must stay in Airdoc's DMZ and that Airdoc has strict quota for GPU usage per programmer/task, the experiment had a total of 3 rounds. A larger number of seeds than 5 would further reduce uncertainty with more statistical power about the relative ranking of MAE and MSE.
- Data skewness: The patients' retinal image dataset, although sizable, is demographically skewed toward younger adults; a more balanced data set would help.
- The CE implementation used simple one-year bins and standard cross-entropy; ordinal regression that treats ages as ordered sets might narrow the performance gap (Niu et al).
- Finally, the study examined just a single modern CNN architecture; results might differ for alternative backbones like ResNet or AlexNet or U-Net (Alom et al).

[Word Count: 3958]

References

Agarwal, Devraj. “A Review of the Math Used in Training a Neural Network.” Level Up Coding, https://levelup.gitconnected.com/a-review-of-the-math-used-in-training-a-neural-network-9b9d5838f272

Ahadi, Sara, et al. "Longitudinal Fundus Imaging and Its Genome-Wide Association Analysis Provide Evidence for a Human Retinal Aging Clock." National Library of Medicine. 17 Apr. 2023. https://pubmed.ncbi.nlm.nih.gov/36975205/ .

Alom, Md Zahangir, et al. “A Holistic Overview of Deep Learning Approach in Medical Imaging.” *Artificial Intelligence Review*, 2022.

"Artificial Neural Networks and its Applications." GeeksforGeeks, 12 July 2025, https://www.geeksforgeeks.org/deep-learning/artificial-neural-networks-and-its-applications/.

"Backpropagation in Neural Network." GeeksforGeeks, 11 July 2025, https://www.geeksforgeeks.org/machine-learning/backpropagation-in-neural-network/.

Brownlee, Jason. "Why Initialize a Neural Network with Random Weights?" Machine Learning Mastery, 15 Aug. 2022, https://machinelearningmastery.com/why-initialize-a-neural-network-with-random-weights/.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT Press.

Google. “TensorFlow, Keras and Deep Learning, without a PhD.” *Google Codelabs*, https://codelabs.developers.google.com/codelabs/cloud-tensorflow-mnist/.

Google for Developers. “Deep Neural Network Models.” *Machine Learning*, Google, https://developers.google.com/machine-learning/recommendation/dnn/softmax

Google for Developers. “Linear Regression: Loss.” *Machine Learning*, Google, https://developers.google.com/machine-learning/crash-course/linear-regression/loss.

Google for Developers. “Neural Networks: Activation Functions.” *Machine Learning*, Google, https://developers.google.com/machine-learning/crash-course/neural-networks/activation-functions

Google for Developers. “Neural Networks: Training Using Backpropagation.” *Machine Learning*, Google, https://developers.google.com/machine-learning/crash-course/neural-networks/backpropagation

"Gradient Descent Algorithm in Machine Learning." GeeksforGeeks, 12 May 2026, https://www.geeksforgeeks.org/gradient-descent-algorithm-and-its-variants/.

Henderson, Peter, et al. "Deep Reinforcement Learning That Matters." *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018. (1)

IBM Data and AI Team. “AI vs. Machine Learning vs. Deep Learning vs. Neural Networks: What’s the Difference?” *IBM*, https://www.ibm.com/think/topics/ai-vs-machine-learning-vs-deep-learning-vs-neural-networks

Ninomiya, Takahiro, et al. "High-Accuracy Retinal Age Prediction via Fundus-Based Multitask Learning Reveals the Effect of Systemic Disease." Communications Medicine, vol. 6, 2026, article 73. https://doi.org/10.1038/s43856-026-01573-y.

Niu, Z., Zhou, M., Wang, L., Gao, X., & Hua, G. (2016). Ordinal regression with multiple output CNN for age estimation. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 4920-4928.

Poplin, R., Varadarajan, A. V., Blumer, K., Liu, Y., McConnell, M. V., Corrado, G. S., Peng, L., & Webster, D. R. (2018). Prediction of cardiovascular risk factors from retinal fundus photographs via deep learning. Nature Biomedical Engineering, 2(3), 158-164.

Rothe, R., Timofte, R., & Van Gool, L. (2018). Deep expectation of real and apparent age from a single image without facial landmarks. International Journal of Computer Vision, 126(2-4), 144-157.

Tan, M., & Le, Q. (2021). EfficientNetV2: Smaller models and faster training. Proceedings of the 38th International Conference on Machine Learning, 10096-10106.

TensorFlow. “tf.nn.softmax_cross_entropy_with_logits.” *TensorFlow*, https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits

Zhu, Z., Shi, D., Pi, G., Zhang, Y., Wang, Y., Sun, Y., Huang, Y., Ding, X., Chen, H., & He, M. (2023). Retinal age gap as a biomarker of aging and age-related disease. Aging Cell, 22(7), e13887.

APPENDIX-I:

1) **Running Script:**

```
36
37  for ((round_index = 1; round_index <= ROUNDS; round_index++)); do
38      round_seed=$((BASE_SEED + round_index - 1))
39      round_output_dir="${SCRIPT_DIR}/${OUTPUT_PREFIX}$(printf '%03d' "$round_index")"
40      mkdir -p "$round_output_dir"
41
42      echo
43      echo "=== Round ${round_index}/${ROUNDS} | seed=${round_seed} | output=$(basename "$round_output_dir")
44
45      for loss in "${LOSSES[@]}"; do
46          loss_output_dir="${round_output_dir}/${loss}-runs"
47          cmd=(/home/anthony/tools/Ares/Ares/shells/run-docker-cmd.sh python3 traintest.py --loss "$loss" --
48
49          echo
50          echo ">>> Running ${loss} for round ${round_index}"
51          printf '$'
52          for part in "${cmd[@]}"; do
53              printf ' %q' "$part"
54          done
55          printf '\n'
56
57          (
58              cd "$SCRIPT_DIR"
59              "${cmd[@]}"
60          )
61      done
62  done
63
64  echo
65  echo "All rounds completed successfully."
```

The multi-seed approach is by design. Prior work on reproducibility in deep learning has shown that single-seed experiments can produce conclusions that reverse under re-seeding, and that aggregate comparisons across seeds are the minimum defensible evidence (Henderson et al., 2018). Three seeds area small numbers but sufficient to separate systematic from random effects when the systematic signal is strong.

All other training hyperparameters are held constant across the nine runs:

Optimizer: Adam

Initial learning rate: 0.006

Batch size: 20

Early-stopping patience: 13 epochs

Weight initialization: Kaiming normal (fan_out, ReLU)

Random seeds: three, distinct across rounds

CE age binning (CE runs only): 69 bins, width 1 year, range [19, 87]

Device: CUDA (NVIDIA GeForce RTX 3090, 24 GB VRAM)

Peak GPU memory: 9.46 GB

2) Implementation

The system is implemented in Python, running inside a training docker env that has PyTorch, torchvision, pandas, numpy, and OpenCV on Airdoc's GPU cluster server. Project Source code is at: https://github.com/MagicalAnt/AirDocAI.git

Here are the code lines for using EfficientNet-V2-M:

```
import torchvision.models as models


…
class MultiTaskInceptionModel(nn.Module):
    def __init__(self, num_age_outputs: int = 1):
        super().__init__()
        self.backbone = models.efficientnet_v2_m(weights=None, progress=True)
            …
```

The training script was adapted so it can switch between different loss modes. Below is the function to select MAE, MSE or CE loss formulation:

```
def get_loss_functions() -> Dict[str, nn.Module]:
    return {
        "mse": nn.MSELoss(),
        "mae": nn.L1Loss(),
        "ce": nn.CrossEntropyLoss(),
    }

….
```

The main function invoking the model training and validation using the loss selection function:

```
def main() -> None:
    args = parse_args()
    …
    model = MultiTaskInceptionModel(
        num_age_outputs=age_binning.num_bins if age_binning is not None else 1
    ).to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr=args.lr)
    …
    loss_functions = get_loss_functions()
```

```
…
with open(log_path, "w", encoding="utf-8", buffering=1) as log_file, \
    open(metrics_csv_path, "w", newline="", encoding="utf-8") as metrics_csv_file:
        …
    for epoch in range(args.epochs):
        …
        train_summary = train_one_epoch(
            model,
            train_loader,
            optimizer,
            device,
            loss_functions,
            loss_config,
            epoch,
            args.epochs,
            log_interval=args.log_interval,
            log_file=log_file,
        )
        val_summary = evaluate(
            model,
            val_loader,
            device,
            loss_functions,
            loss_config,
            phase="val",
        )
```

APPENDIX-II: sample of a run's epoch_metrics.csv (round 1, CE)

epoch,split,avg_loss,age_abs_error_mean,age_mse,lr,duration_seconds
1,train,3.940821,12.701809,290.882278,0.00600000,625.94
1,val,3.872523,13.557048,332.064952,0.00600000,142.25
2,train,3.887990,12.717238,293.525951,0.00600000,631.43
2,val,3.836307,13.152928,311.624407,0.00600000,141.30
3,train,3.881893,12.801776,298.415842,0.00600000,626.54
3,val,3.871078,12.076882,267.431377,0.00600000,138.73
4,train,3.878724,12.844699,300.717250,0.00600000,618.34
4,val,3.867040,12.076882,267.431377,0.00600000,136.73
5,train,3.873807,12.690013,294.384889,0.00600000,614.65
5,val,3.864397,13.003215,309.212213,0.00600000,136.06
6,train,3.872164,12.756717,297.052818,0.00600000,648.85
6,val,3.864840,13.003691,309.223459,0.00600000,300.22
7,train,3.872156,12.829597,299.832619,0.00600000,640.07
7,val,3.862662,12.503152,287.327419,0.00600000,136.69
8,train,3.872328,12.809112,298.998914,0.00600000,638.76
8,val,3.863930,12.076882,267.431377,0.00600000,138.56
9,train,3.873131,12.900095,302.921206,0.00600000,644.74
9,val,3.863120,12.076882,267.431377,0.00600000,141.69
10,train,3.872076,12.823572,300.084255,0.00600000,652.23
10,val,3.863681,12.076882,267.431377,0.00600000,136.06
11,train,3.869912,12.389484,280.955078,0.00090000,622.95
11,val,3.862914,12.076882,267.431377,0.00090000,136.30
12,train,3.869487,12.249891,274.463437,0.00090000,617.88
12,val,3.862684,13.003691,309.223459,0.00090000,136.75
13,train,3.869356,12.615835,290.649669,0.00090000,624.11
13,val,3.862507,13.003691,309.223459,0.00090000,134.18
14,train,3.869341,13.070232,310.647185,0.00090000,625.45
14,val,3.862442,13.003691,309.223459,0.00090000,135.66
15,train,3.869302,13.070232,310.647185,0.00090000,613.10
15,val,3.862338,13.003691,309.223459,0.00090000,135.82
16,train,3.869306,13.070232,310.647185,0.00090000,617.60
16,val,3.862312,13.003691,309.223459,0.00090000,134.76

APPENDIX-III: sample of a run's training_log.txt

training started

label csv: /mnt/workdir/xiongjianhao/Project/Anthony-2025Dec/data/val_routine_eGFR_v0.4.0.csv

image folder: /mnt/workdir/xiongjianhao/Project/Anthony-2025Dec/data/images/val

missing image log: /home/anthony/development/agegender/outputs001/mae-runs/missing_images.txt

train samples: 19621

val samples: 4204

test samples: 4206

device: cuda

epoch 001 started

epoch 001 step 00001/00982: loss=58.7051 | age_abs_error_mean=58.7051 | age_mse=4772.2703 |
age_weighted_loss=58.7051 | fail_rate=0.0000 | elapsed_min=0.09

epoch 001 step 00020/00982: loss=19.0441 | age_abs_error_mean=19.0441 | age_mse=999.4308 |
age_weighted_loss=19.0441 | fail_rate=0.0000 | elapsed_min=0.24

epoch 001 step 00040/00982: loss=16.5193 | age_abs_error_mean=16.5193 | age_mse=688.5676 |
age_weighted_loss=16.5193 | fail_rate=0.0000 | elapsed_min=0.72

...

epoch 077 step 00960/00982: loss=3.5393 | age_abs_error_mean=3.5393 | age_mse=26.6388 |
age_weighted_loss=3.5393 | fail_rate=0.0000 | elapsed_min=10.52

epoch 077 step 00980/00982: loss=3.5374 | age_abs_error_mean=3.5374 | age_mse=26.5823 |
age_weighted_loss=3.5374 | fail_rate=0.0000 | elapsed_min=10.76

epoch 077 train: loss=3.5376 | age_abs_error_mean=3.5376 | age_mse=26.5831 | age_weighted_loss=3.5376 |
samples=19610 | batches=982 | skipped_batches=0 | fail_rate=0.0000 | duration_min=10.78 | max_memory_gb=9.46

epoch 077 val: loss=4.2825 | age_abs_error_mean=4.2825 | age_mse=38.3410 | age_weighted_loss=4.2825 |
samples=4198 | batches=211 | skipped_batches=0 | fail_rate=0.0000 | duration_min=3.14 | max_memory_gb=9.46

final test: loss=4.0516 | age_abs_error_mean=4.0516 | age_mse=32.6441 | age_weighted_loss=4.0516 |
samples=4204 | batches=211 | skipped_batches=0 | fail_rate=0.0000 | duration_min=4.95 | max_memory_gb=9.46

APPENDIX-IV: samples of val_predicitons.csv

Sample 1: Round 1, MAE

true_age,predicted_age,absolute_error,squared_error

39.1096,36.6757,2.4339,5.9238

48.3425,54.7242,6.3817,40.7265

36.4411,39.5432,3.1021,9.6231

22.8904,25.5635,2.6731,7.1456

62.4384,62.0133,0.4250,0.1806

50.3370,43.5565,6.7805,45.9754

38.3671,34.3848,3.9823,15.8590

29.0575,31.2280,2.1704,4.7107

28.7288,29.2490,0.5202,0.2706

22.3699,24.0651,1.6953,2.8739

...

31.9644,30.6470,1.3174,1.7356

43.5452,40.1836,3.3616,11.3004

32.5562,26.7431,5.8131,33.7916

Sample 2: Round 1, CE

true_age,predicted_age,absolute_error,squared_error

39.1096,32.0000,7.1096,50.5463

48.3425,32.0000,16.3425,267.0761

36.4411,32.0000,4.4411,19.7233

22.8904,32.0000,9.1096,82.9846

62.4384,32.0000,30.4384,926.4934

50.3370,32.0000,18.3370,336.2451

38.3671,32.0000,6.3671,40.5403

29.0575,32.0000,2.9425,8.6581

28.7288,32.0000,3.2712,10.7010

22.3699,32.0000,9.6301,92.7395

….

31.9644,32.0000,0.0356,0.0013

43.5452,32.0000,11.5452,133.2917

32.5562,32.0000,0.5562,0.3093

APPENDIX-V: sample of a run's test_predictions.csv

true_age,predicted_age,absolute_error,squared_error

28.8767,31.0550,2.1782,4.7447

58.2164,54.0198,4.1966,17.6114

64.7096,56.8956,7.8140,61.0589

36.0959,32.5004,3.5955,12.9274

39.2055,38.2320,0.9735,0.9477

34.7945,36.0494,1.2548,1.5746

41.4219,46.4433,5.0213,25.2139

39.5616,35.2988,4.2628,18.1717

69.6055,65.0066,4.5988,21.1492

43.0411,53.0214,9.9803,99.6066

43.5205,41.7184,1.8022,3.2478

39.9699,39.0068,0.9631,0.9275

46.4000,43.7655,2.6345,6.9404

...

32.1233,39.2158,7.0925,50.3035

61.7644,62.9639,1.1995,1.4389

36.0247,39.6688,3.6442,13.2801

APPENDIX-VI: Run data files

learning data files:

https://magicalant.github.io/docs/round_1_raw.csv,

https://magicalant.github.io/docs/round_2_raw.csv,

https://magicalant.github.io/docs/round_3_raw.csv,

summary: runs_data.csv

round	loss	mae_years	collapsed	total_epochs
1	ce	12.0396	TRUE	16
2	ce	4.7889	FALSE	64
3	ce	4.8721	FALSE	77
1	mae	4.0516	FALSE	77
2	mae	4.6066	FALSE	52
3	mae	4.2821	FALSE	76
1	mse	4.3945	FALSE	52
2	mse	4.4137	FALSE	46
3	mse	4.5546	FALSE	62

Below is a partial sample of round_2_raw.csv;

Epoch,MAE train,MSE train,CE train,MAE val,MSE val,CE val

1,11.915268,11.558715,12.7785,11.052902,14.136786,12.622234

2,11.199353,11.174522,11.439935,32.588918,28.969889,9.459646

3,11.126631,11.18585,9.105695,24.702325,12.97453,10.453503

4,11.033772,11.232505,9.027184,10.939565,12.009535,8.264166

5,10.905755,11.221017,9.28331,10.586268,11.031643,9.014717

6,10.07927,11.18141,8.951693,9.557696,11.733667,8.613112

7,8.896953,11.198237,8.589456,53.187899,11.143129,9.906588

8,7.811789,11.184629,8.343977,11.424689,11.192486,10.435887

9,7.11001,10.431311,8.132426,6.77609,10.917213,9.062122

10,6.659283,8.601329,7.935717,5.97319,7.734848,8.024976

....

61,,,3.900446,,,4.723142

62,,,3.918476,,,4.72117

63,,,3.920527,,,4.718525

64,,,3.922223,,,4.743259